

SQLAlchemy ORM

Guia de paralelos: SQL puro ↔ SQLAlchemy · Python

■ Setup — Configuração inicial

OPERAÇÃO	SQL puro	SQLAlchemy ORM
Conexão / Sessão	<pre>-- não existe 'conexão' explícita -- no SQL puro do dia a dia conn = sqlite3.connect('test.db') cursor = conn.cursor()</pre> ↑ Session = unidade de trabalho do ORM (parecida com uma transação)	<pre>engine = create_engine('sqlite:///test.db') Base = declarative_base() SessionLocal = sessionmaker(bind=engine) db = SessionLocal()</pre>
Criar tabela	<pre>CREATE TABLE users (id INTEGER PRIMARY KEY, name VARCHAR(100), email VARCHAR(100) UNIQUE);</pre> ↑ O ORM gera o DDL automaticamente a partir da classe	<pre>class User(Base): __tablename__ = 'users' id = Column(Integer, primary_key=True) name = Column(String(100)) email = Column(String(100), unique=True) Base.metadata.create_all(bind=engine)</pre>

■ INSERT — Inserir registros

OPERAÇÃO	SQL puro	SQLAlchemy ORM
Inserir 1 registro	<pre>INSERT INTO users (name, email) VALUES ('Ana', 'ana@email.com');</pre> ↑ db.add() agenda a inserção; commit() executa no banco	<pre>novo = User(name='Ana', email='ana@email.com') db.add(novo) db.commit()</pre>
Inserir vários	<pre>INSERT INTO users (name, email) VALUES ('Bob', 'bob@email.com'), ('Carol', 'carol@email.com');</pre> ↑ add_all() recebe uma lista de objetos	<pre>db.add_all([User(name='Bob', email='bob@email.com'), User(name='Carol', email='carol@email.com'),]) db.commit()</pre>

■ SELECT — Consultar registros

OPERAÇÃO	SQL puro	SQLAlchemy ORM
SELECT *	<pre>SELECT * FROM users;</pre> ↑ .all() retorna lista de objetos User	<pre>db.query(User).all()</pre>
SELECT com WHERE	<pre>SELECT * FROM users WHERE email = 'ana@email.com';</pre> ↑ Use == (não =) dentro do filter()	<pre>db.query(User) .filter(User.email == 'ana@email.com') .all()</pre>

OPERAÇÃO	SQL puro	SQLAlchemy ORM
Buscar por PK (first)	<pre>SELECT * FROM users WHERE id = 1;</pre> ↑ .first() retorna None se não achar; .get() também	<pre>db.query(User).filter(User.id == 1).first() # OU forma idiomática: db.get(User, 1)</pre>
Múltiplos filtros (AND)	<pre>SELECT * FROM users WHERE name = 'Ana' AND email LIKE '%@email.com';</pre> ↑ Múltiplos args no filter() = AND implícito	<pre>db.query(User) .filter(User.name == 'Ana', User.email.like('%@email.com')).all()</pre>
Filtro OR	<pre>SELECT * FROM users WHERE name='Ana' OR name='Bob';</pre> ↑ Use or_() e and_() do sqlalchemy para lógica booleana	<pre>from sqlalchemy import or_ db.query(User) .filter(or_(User.name == 'Ana', User.name == 'Bob')).all()</pre>
ORDER BY + LIMIT	<pre>SELECT * FROM users ORDER BY name ASC LIMIT 5;</pre> ↑ .desc() para decrescente	<pre>db.query(User) .order_by(User.name.asc()) .limit(5) .all()</pre>
Contar registros	<pre>SELECT COUNT(*) FROM users;</pre> ↑ Retorna int direto	<pre>db.query(User).count()</pre>
Selecionar colunas	<pre>SELECT name, email FROM users;</pre> ↑ Retorna lista de tuplas NamedTuple, não objetos User	<pre>db.query(User.name, User.email).all()</pre>

■ UPDATE — Atualizar registros

OPERAÇÃO	SQL puro	SQLAlchemy ORM
Atualizar um objeto	<pre>UPDATE users SET name = 'Ana Silva' WHERE id = 1;</pre> ↑ O ORM detecta a mudança no objeto — não precisa de .update()	<pre>usuario = db.get(User, 1) usuario.name = 'Ana Silva' db.commit()</pre>
Atualizar em massa	<pre>UPDATE users SET name = 'Anonimo' WHERE email LIKE '%teste%';</pre> ↑ .update() em massa não carrega objetos na memória (mais performático)	<pre>db.query(User) .filter(User.email.like('%teste%')) .update({'name': 'Anonimo'}) db.commit()</pre>

■ DELETE — Remover registros

OPERAÇÃO	SQL puro	SQLAlchemy ORM
Deletar um registro	<pre>DELETE FROM users WHERE id = 1;</pre> ↑ Precisa buscar o objeto antes de deletar	<pre>usuario = db.get(User, 1) db.delete(usuario) db.commit()</pre>

OPERAÇÃO	SQL puro	SQLAlchemy ORM
Deletar em massa	<pre>DELETE FROM users WHERE email LIKE '%spam%';</pre> ↑ <code>.delete()</code> em massa também não carrega objetos na memória	<pre>db.query(User) .filter(User.email.like('%spam%')) .delete() db.commit()</pre>

■ Transações & rollback

OPERAÇÃO	SQL puro	SQLAlchemy ORM
Commit / Rollback	<pre>BEGIN; INSERT INTO users ... COMMIT; -- ou: ROLLBACK;</pre> ↑ Sempre use try/except em operações de escrita	<pre>try: db.add(User(name='X', email='x@x.com')) db.commit() except: db.rollback() raise</pre>
Refresh objeto	<pre>-- re-ler dados do banco: SELECT * FROM users WHERE id = ?</pre> ↑ Atualiza o objeto Python com os dados mais recentes do banco	<pre>db.refresh(usuario)</pre>

■ Dicas de nível profissional

OPERAÇÃO	SQL puro	SQLAlchemy ORM
Feche a sessão	Sempre feche a sessão após uso: ↑ Evita connection leak — crítico em APIs com muitas requisições	<pre>db.close() # ou use context manager: with SessionLocal() as db: ...</pre>
Session como dependência	<pre>-- padrão em FastAPI: yield db (Depends)</pre> ↑ Dependency Injection garante que a sessão sempre seja fechada	<pre>def get_db(): db = SessionLocal() try: yield db finally: db.close()</pre>
SQL bruto quando necessário	<pre>SELECT * FROM users WHERE id = :id</pre> ↑ Use <code>text()</code> para queries complexas que o ORM não expressa bem	<pre>from sqlalchemy import text db.execute(text('SELECT * FROM users WHERE id=:id'), {'id': 1}).fetchall()</pre>